

✓ .NET Interview Questions (3–5 Years) – With Answers

◆ Basic & Core .NET

1. **What is .NET Core?**
Cross-platform, high-performance framework for building apps.
 2. **Difference: .NET Core vs .NET Framework?**
Core = cross-platform, Framework = Windows only.
 3. **CLR?**
Common Language Runtime executes .NET code.
 4. **CTS?**
Common Type System defines data types.
 5. **CLS?**
Common Language Specification ensures language interoperability.
-

◆ C# Concepts

6. **OOP Principles?**
Encapsulation, Inheritance, Polymorphism, Abstraction.
 7. **Interface vs Abstract Class?**
Interface = only declaration, Abstract = can have implementation.
 8. **What is Polymorphism?**
Same method, different behavior.
 9. **What is Encapsulation?**
Data hiding using classes.
 10. **What is Inheritance?**
Reusing code from parent class.
-

◆ ASP.NET Core

11. **Middleware?**
Pipeline components handling request/response.
 12. **Dependency Injection?**
Injecting dependencies instead of creating manually.
 13. **Service Lifetimes?**
Scoped, Singleton, Transient.
 14. **Routing?**
Maps URL to controller/action.
 15. **Model Binding?**
Maps request data to model.
-

◆ Web API

- 16. **What is REST API?**
Stateless communication using HTTP.
 - 17. **HTTP Methods?**
GET, POST, PUT, DELETE, PATCH.
 - 18. **PUT vs PATCH?**
PUT = full update, PATCH = partial update.
 - 19. **Status Codes?**
200 OK, 404 Not Found, 500 Error.
 - 20. **JWT?**
Token-based authentication.
-

◆ Entity Framework / Database

- 21. **Entity Framework Core?**
ORM to interact with database.
 - 22. **LINQ?**
Query language in C#.
 - 23. **IEnumerable vs IQueryable?**
IEnumerable = in-memory, IQueryable = DB query.
 - 24. **First vs FirstOrDefault?**
First throws error, FirstOrDefault returns null.
 - 25. **Migrations?**
Database schema updates.
-

◆ Advanced Concepts

- 26. **SOLID Principles?**
5 design principles for clean code.
 - 27. **Repository Pattern?**
Abstract data access logic.
 - 28. **Unit of Work?**
Manages DB transactions.
 - 29. **CQRS?**
Separate read/write operations.
 - 30. **Clean Architecture?**
Layered structure for maintainability.
-

◆ Security

- 31. **Authentication vs Authorization?**
Auth = login, Authorization = access control.

- 32. **OAuth?**
Token-based authorization.
 - 33. **CORS?**
Allows cross-origin requests.
 - 34. **CSRF?**
Unauthorized request attack.
 - 35. **XSS?**
Script injection attack.
-

◆ Performance & Logging

- 36. **Caching?**
Store data for faster access.
 - 37. **Logging?**
Tracking app behavior.
 - 38. **Serilog?**
Logging library.
 - 39. **NLog?**
Logging framework.
 - 40. **Health Checks?**
Check app status.
-

◆ Async & Multithreading

- 41. **async/await?**
Non-blocking execution.
 - 42. **Task?**
Represents async operation.
 - 43. **Thread?**
Execution path.
 - 44. **Deadlock?**
Threads waiting forever.
 - 45. **Lock?**
Prevents concurrent access.
-

◆ API & Architecture

- 46. **Microservices?**
Small independent services.
- 47. **Docker?**
Containerization tool.
- 48. **API Gateway?**
Single entry point.

- 49. **gRPC?**
High-performance communication.
 - 50. **SignalR?**
Real-time communication.
-

◆ Azure & DevOps

- 51. **Azure App Service?**
Host web apps.
 - 52. **Azure Functions?**
Serverless computing.
 - 53. **CI/CD?**
Automated deployment.
 - 54. **Git?**
Version control.
 - 55. **Branching?**
Code version management.
-

◆ Testing

- 56. **Unit Testing?**
Test small units.
 - 57. **xUnit?**
Testing framework.
 - 58. **Moq?**
Mocking framework.
 - 59. **Integration Testing?**
Test full system.
 - 60. **TDD?**
Test-driven development.
-

◆ SQL Concepts

- 61. **Normalization?**
Remove redundancy.
- 62. **Denormalization?**
Improve performance.
- 63. **Indexing?**
Speed up queries.
- 64. **ACID?**
Transaction rules.
- 65. **Stored Procedure?**
Predefined SQL logic.

◆ ADO.NET & Dapper

- 66. **ADO.NET?**
Direct DB access.
- 67. **Dapper?**
Lightweight ORM.
- 68. **Connection Pooling?**
Reuse DB connections.
- 69. **DataReader?**
Read-only data access.
- 70. **DataSet?**
In-memory data.

◆ API Features

- 71. **Pagination?**
Limit records.
- 72. **Filtering?**
Select specific data.
- 73. **Sorting?**
Order data.
- 74. **Versioning?**
API versions.
- 75. **Rate Limiting?**
Limit API calls.

◆ Frontend Integration

- 76. **JSON?**
Data format.
- 77. **Serialization?**
Object → JSON.
- 78. **Deserialization?**
JSON → Object.
- 79. **Swagger?**
API documentation.
- 80. **Postman?**
API testing tool.

◆ Memory & Performance

81. Garbage Collection?

Memory cleanup.

82. IDisposable?

Release resources.

83. using keyword?

Auto dispose.

84. Heap vs Stack?

Memory areas.

85. Boxing/Unboxing?

Type conversion.

◆ **Advanced Topics**

86. Reflection?

Inspect code at runtime.

87. Middleware Pipeline?

Request flow.

88. Kestrel?

Web server.

89. Minimal API?

Lightweight API.

90. Blazor?

C# frontend framework.

◆ **Best Practices**

91. Error Handling?

Try-catch, middleware.

92. Validation?

Model validation.

93. Code Optimization?

Improve performance.

94. Secure API?

Use HTTPS, JWT.

95. Clean Code?

Readable, maintainable.

◆ **Real-world**

96. How to improve API performance?

Caching, indexing.

97. How to handle exceptions globally?

Middleware.

98. **How to secure API?**

JWT, OAuth.

99. **How to deploy app?**

Azure / IIS.

100. **Best practices in .NET?**

SOLID, DI, clean architecture.